

MATRIZ DE OPTIMIZACION DE COSTOS

Proyecto Final - Asistente de Cartas Oferta

Autora: Virginia Rangel

1. La tarea de IA en este proyecto

El sistema usa la IA en un único punto del flujo: el nodo “Message a model”, que lee el email del proveedor y extrae 4 datos (nombre y apellido, cargo, dirección fiscal, email), además de revisar consistencia y terminología. Es una tarea de lectura corta y extracción estructurada, no requiere razonamiento complejo, generación de texto largo, ni comprensión de documentos densos.

Por esa razón, el modelo elegido fue gpt-4o-mini, el más económico de la familia de OpenAI, en vez de un modelo de mayor capacidad como gpt-4o o GPT-4.1.

2. Comparativa de modelos para esta tarea

Precios vigentes de la API de OpenAI (septiembre 2026, por millón de tokens):

Modelo	Precio entrada	Precio salida	¿Apto para esta tarea?
gpt-4o-mini (elegido)	USD 0.15	USD 0.60	Si - la tarea es simple, no necesita más capacidad
gpt-4o	USD 2.50	USD 10.00	Sobredimensionado - 16x más caro sin beneficio real
GPT-4.1	USD 2.00 - 5.00	USD 8.00 - 15.00	Sobredimensionado para extraer 4 campos

3. Calculo del costo real por ejecución

Estimando el prompt completo (instrucciones fijas + email del proveedor) en ~350 tokens de entrada, y una respuesta JSON de ~100 tokens de salida:

Modelo	Costo entrada	Costo salida	Costo por ejecucion	Costo x 20 cartas/mes
gpt-4o-mini	USD 0.000052	USD 0.000060	USD 0.000112	USD 0.0022
gpt-4o (comparacion)	USD 0.000875	USD 0.001000	USD 0.001875	USD 0.0375

Con gpt-4o-mini, procesar 20 cartas oferta por mes cuesta aproximadamente USD 0.0022 — una fracción de centavo. Usar gpt-4o para la misma tarea costaría 16.7 veces más, sin ninguna mejora real en el resultado, porque la tarea (extraer 4 campos de un email corto) no exige más capacidad de la que gpt-4o-mini ya tiene.

4. ¿Por qué no se usa la Message Batches API?

La Batch API ofrece 50% de descuento, pero introduce una demora de hasta 24 horas antes de recibir la respuesta, una condición incompatible con este flujo: la extracción de datos tiene que completarse en el momento, para que la fila pase de inmediato al punto de validación humana (HITL) y el comprador pueda revisar y aprobar sin esperar un día entero solo para llegar a ese paso, sumando una demora innecesaria a un trámite que ya tiene varios pasos administrativos después (carga en herramienta interna de la empresa, firma del proveedor).

Es una decisión de diseño consciente: el ahorro del 50% no justifica sacrificar la respuesta en tiempo real que el proceso de compras necesita.

5. ¿Aplica Prompt Caching?

Parcialmente. El bloque de instrucciones fijas del prompt (rol, objetivo, reglas, formato de salida) es idéntico en cada ejecución, exactamente el tipo de contenido que Prompt Caching está pensado para optimizar.

Escenario	Costo del prefijo fijo (por llamada)	Ahorro x 20 cartas/mes
Sin Prompt Caching	USD 0.000037	—
Con Prompt Caching (-50% en OpenAI)	USD 0.000019	USD 0.000375

El ahorro real, a este volumen (20 cartas/mes), es de apenas USD 0.000375 mensuales, prácticamente cero en términos absolutos. Por eso no se implementó en este proyecto: la complejidad técnica de activar el cache_control no se justifica frente a un ahorro tan chico a este volumen. Si el proceso escalara a cientos de cartas por día, la decisión sería distinta.

6. Conclusión

La decisión de costos de este proyecto no fue elegir la herramienta más sofisticada disponible, sino la más adecuada al tamaño real de la tarea: un modelo económico para una extracción simple, sin Batch (porque el flujo necesita respuesta inmediata) y sin Prompt Caching (porque el volumen actual no justifica la complejidad). El costo total de IA de todo el sistema, a 20 cartas por

mes, es de centavos de dólar, una automatización que se paga sola muchas veces sobre el tiempo que ahorra en la carga manual de datos.